

Ex01

- * Allgemeines (Laden, Pfad)
- † `data = np.loadtxt(file, skiprows=1, delimiter="|", comments="#", unpack=True)`
- ‡ `np.savetxt('DataOut.txt', (x, y, z), delimiter=',')`
- § einfacher Plot mit “scatter”
- ¶ Formatting
- || Dateien wieder speichern

Ex02

- * `import numpy as np`
`import matplotlib.pyplot as plt`
- † einfacher Plot mit “plot”
- ‡ Histogramm
- § Funktion zur Bestimmung der Frequenz “get_frequency”

Ex03

- * einfache Funktionen
- † Mittelwert μ und Standardabweichung σ ($\text{ddof}=0 \Rightarrow \frac{1}{N}$, $\text{ddof}=1 \Rightarrow \frac{1}{N-1}$)
- ‡ Plot mit `linspace` und nochmals ein Plot mit Balken
- § die Aufgabe mit den Coins
- ¶ `n = 10` # Anzahl Versuche
`p = 0.3` # Erfolgswahrscheinlichkeit
`N = 10000` # Anzahl Stichproben
- `x = np.random.binomial(n, p, N)`
`# x = random.choice([0,1,2], p=[0.3,0.2,0.5],size=n)`
`plt.hist(x, bins=np.arange(-0.5, n+1.5, 1), density=True)`
- || $\mu = np$ und $\sigma^2 = np(1-p)$ theoretische Werte oder einfach den Versuch tausendmal ausführen und dann den Mittelwert nehmen

Ex04

- * Plotten mit `logspace`
- † Unsicherheit des Mittelwertes

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{N}}$$

- ‡ Gaußsche Fehlerfortpflanzung
- § Kovarianz/Korrelationskoeffizienten/Autokovarianz

¶
$$r = \frac{\text{cov}(x, y)}{\sigma_x \sigma_y} \quad -1 \leq r \leq 1$$

−1 perfekte negative Korrelation 0 keine Korrelation 1 perfekte korrelation

||
$$\sigma_f^2 = \sum_i \left(\frac{\partial f}{\partial x_i} \right)^2 \sigma_i^2 + 2 \sum_{i < j} \frac{\partial f}{\partial x_i} \frac{\partial f}{\partial x_j} \text{Cov}(x_i, x_j)$$

Ex05

- * Autokovarianz als Funktion der Zeit
- † Weißes Rauschen: keine Korrelation

Ex06

* Fouriertransformation
† Nyquist-Frequenz
‡ `np.argmax()`
`idx = np.argsort(x)[-3:]`
`from scipy.signal import find_peaks`
§ Peaks finden

`from scipy.signal import find_peaks`

`peaks, _ = find_peaks(y)`

`// Drei höchste Peaks auswählen`
`idx = peaks[np.argsort(y[peaks])[-3:]][:, -1]`

`print(idx) // Indizes der drei höchsten Peaks`
`print(y[idx]) // Peak-Höhen`

Ex07

* Moving Average
† Glättet das Signal und entfernt schnelles Rauschen.
‡ Filtern im Frequenzraum
§ Signal Reconstruction (Signalrekonstruktion)
¶ Hochpass und Tiefpass

- Tiefpass: Lässt tiefe Frequenzen durch und unterdrückt hohe Frequenzen.
 - Verwendung: Wenn hochfrequentes Rauschen entfernt oder das Signal geglättet werden soll.
 - Beispiele: Sensorrauschen entfernen, Messdaten glätten.
- Hochpass: Lässt hohe Frequenzen durch und unterdrückt tiefe Frequenzen.
 - Verwendung: Wenn langsame Trends oder Offsets entfernt und schnelle Änderungen hervorgehoben werden sollen.
 - Beispiele: Kanten in Bildern erkennen, plötzliche Signaländerungen analysieren.

|| ein guter Plot der `fft` macht “noisy_pulse”

Ex08

* Wahrscheinlichkeitsmassenfunktion
† Mode / Median / Mediansort / Mean
‡ FWHM:

$$FWHM = 2\sigma\sqrt{2\log 2}$$

§ Momente

* Achtung: Es gibt einen Unterschied zwischen dem Moment und dem zentralen Moment.

```
import numpy as np

def central_moment(x, k):
    x = np.asarray(x)
    mu = np.mean(x)
    return np.mean((x - mu)**k)

def moment(data, n):
    return np.sum(data**n) / len(data)
```

- ¶ Das dritte zentrale Moment ist für eine symmetrische Verteilung gleich 0 und für eine asymmetrische Verteilung ungleich 0.

Ex09

- * Gaußsche Normalverteilung

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2} \frac{(x-\mu)^2}{\sigma^2}}$$

- † Zentraler Grenzwertsatz

- Beschreibt, dass sich die Summe bzw. der Mittelwert vieler unabhängiger und identisch verteilter Zufallsvariablen einer Normalverteilung annähert.
- Gilt unabhängig von der ursprünglichen Verteilung der Daten (bei genügend großer Stichprobe und endlicher Varianz).
- Wichtig: Die Daten müssen unabhängig und identisch verteilt (i.i.d.) sein.
- Die Art der Stichprobenziehung ist entscheidend, damit der Satz anwendbar ist.

Ex10

- * Lineares Fitten

```
a, cov = np.polyfit(
    x,          # x-Daten
    y,          # y-Daten
    deg=1,      # Grad des Polynoms (1 = Gerade)
    w=None,     # Gewichte (z.B. 1/sigma)
    cov=True    # Kovarianzmatrix zurückgeben
)
```

- † Chi-Quadrat-Funktion

$$\chi^2 = \sum_i \left(\frac{y_i - f(x_i)}{\sigma_i} \right)^2$$

- Misst die Abweichung zwischen Messdaten und Modell.
- $\chi_{\text{red}}^2 \approx 1$: guter Fit.
- $\chi_{\text{red}}^2 \gg 1$: schlechter Fit oder Unsicherheiten zu klein.
- $\chi_{\text{red}}^2 \ll 1$: Unsicherheiten zu groß oder Modell passt "zu gut".

Reduziertes Chi-Quadrat (wenn es sehr viel Datenpunkte hat):

$$\chi_{\text{red}}^2 = \frac{\chi^2}{N - \text{len}(a)}$$

- N : Anzahl Messpunkte.
- a : Anzahl der Fitparameter.

- ‡ pcolormesh

- Erstellt eine 2D-Farbkarte für Matrixdaten.
- Geeignet für Heatmaps und Intensitätsverteilungen.

Ex11

- * Gradient and descent

- † Nichtlinearer Fit (scipy.optimize.curve_fit)

```
def f(x, a, b, c):
    return a * b * c

from scipy.optimize import curve_fit
popt, pcov = curve_fit(
    f,                # Modellfunktion
    xdata,            # x-Daten
    ydata,            # y-Daten
    p0=None,          # Startwerte der Parameter
    sigma=None,       # Messunsicherheiten
    absolute_sigma=False/True,
)
a, b, c = popt
```

In popt wären dann alle Fitparameter

Ex12

* Datenverarbeitung mit pandas

† Regressions

* linear

† polynomial

‡ Ridge_regr

‡ beste Hyperparameter finden

```
from sklearn.model_selection import GridSearchCV
```

Ex13

* Logistische Regression

† MLP Classifier

‡ accuracy_score

§ Propagation durch ein neuronales Netz

¶ pickle

Ex14

* Listen bearbeiten / Bereinigung

† SimpleImputer

‡ One-Hot-Kodierung (wenn Daten Kategorisch sind)

§ Mean-Kodierung

¶ KNN und Decision Tree

|| Wann muss man einen Scaler verwenden?

- Scaler verwenden bei Verfahren, die Abstände oder Gradienten verwenden:
 - KNN
 - Logistische Regression
 - MLP / Neuronale Netze
 - SVM
- Kein Scaler nötig bei:
 - Decision Tree
 - Random Forest

** Beispiel code

```

tree = DecisionTreeClassifier(random_state=42)

X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# Daten Aufteilen in Trainings- und Testdaten
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Daten skalieren

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # falls nötig
X_test_scaled = scaler.transform(X_test)

tree.fit(X_train, y_train) #falls scaler angewendet natürlich ..._scaled

y_pred_train = tree.predict(X_train)
y_pred_test = tree.predict(X_test)

accuracy_train = accuracy_score(y_train, y_pred_train)
accuracy_test = accuracy_score(y_test, y_pred_test)

print("Trainingsgenauigkeit:", accuracy_train)
print("Testgenauigkeit:", accuracy_test)

```

Andere Sachen

* Messung = $\underbrace{x}_{\text{tatsächlich}} \cdot \underbrace{G}_{\text{Gain}}$

† $\sigma^2 = \text{PSD} \cdot \underbrace{\Delta T}_{\text{intervall}}$

‡

	Erwartungswert	std
$\text{Bin}(N, p)$	$N \cdot p$	$\sqrt{Np(1-p)}$
$\text{poisson}(\lambda)$	λ	$\sqrt{\lambda}$
$\text{Normal}(\mu, \sigma^2)$	μ	σ

§ Poisson $\lambda :=$ Erwartungswert & $k :=$ Ereignisse Werte von 0 bis 1 (Wahrscheinlichkeit).

$$p(k, \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}$$

¶ Wie man die Matrixen verarbeitet

* Relu: $f(x) = \max(0, x)$

† tanh: $f(x) = \tanh(x)$

‡ sigmoid $f(x) = \frac{1}{1+e^{-x}}$

§ Pooling

* Standart

$$\begin{pmatrix} a_1 & a_2 \\ a_3 & a_4 \end{pmatrix} \rightarrow \begin{pmatrix} \max(a_{11}, a_{1,2}, a_{13}a_{14}) & \dots \\ \dots & \dots \end{pmatrix}$$

† average pooling

‡ min pooling

§ l2 pooling $:= \sqrt{\sum_i x_i^2}$